

SPRINT 07 – DEPLOY COM API REAL

Nesta etapa final, o foco foi construir em **React** uma nova área da plataforma **Iara Games** que consome uma **API real** e publicá-la online. A equipe desenvolveu a **tela de login e cadastro**, integrada à API pública **DummyJSON**, conectando o portal a uma fonte de dados externa e encerrando a jornada de prototipação, UX, UI, front-end e integrações das fases anteriores. Aproveitando esta fase, a equipe também **converteu todo o projeto para React** e **migrou a hospedagem do GitHub Pages para a Vercel**, reunindo todas as páginas da plataforma em uma única aplicação e um único endereço.

Integrantes:

• Ingrid Silva de Lima - rm570149

• Larissa Cristina Benedito - rm570970

• Mayla Mayumi Motobe.- rm571213

Repositório GitHub: https://github.com/mewmewdevart/IaraGamesStore

Link do deploy (Vercel): https://iaragames-store.vercel.app/

Link da área desenvolvida (Login e cadastro): https://iaragames-store.vercel.app/login.html

1. Funcionalidades Implementadas:

O objetivo desta etapa foi desenvolver uma **área de autenticação funcional, conectada a uma API externa e publicada online**, que serve como porta de entrada do jogador na loja.

A tela foi construída em **React** (empacotada via **Vite**), fica na rota `/login.html` e reúne dois modos na mesma interface: **Entrar** e **Cadastre-se**. Ela consome os endpoints públicos da **DummyJSON**, um serviço de dados simulados próprio para testes e demonstrações.

A interface oferece as seguintes funcionalidades:

- **Login:** campos obrigatórios de usuário e senha; um login válido exibe a saudação “Olá, [nome]!” com o nome devolvido pela API;
- **Validação de credenciais:** senha ou usuário incorretos exibem a mensagem “Usuário ou senha inválidos.”;
- **Cadastro:** campos de usuário, e-mail e senha, com validação inicial feita pelo navegador;
- **Modal de boas-vindas:** após o cadastro, mostra o nome retornado pela API e avisa que a conta é demonstrativa; fecha pelo botão “Entendi”, por clique fora ou pela tecla Escape;
- **Estado de carregamento:** durante a requisição, o botão exibe “Aguarde...” e fica desabilitado, evitando envios repetidos;
- **Mensagens de erro:** falhas de conexão ou respostas inesperadas aparecem no próprio formulário.

[Inserir print – Tela de login da Iara Games]

[Inserir print – Saudação “Olá, Emily!” após login válido]

[Inserir print – Modal de boas-vindas após o cadastro]

2. Integração com a API DummyJSON

Comunicação entre a interface React e a API

- **Requisições HTTP:** a tela envia os dados direto do navegador para `https://dummyjson.com`, usando `fetch`, o método `POST` e o cabeçalho `Content-Type: application/json`.
- **Endpoints consumidos:** cada ação da tela chama um endpoint diferente da API, conforme a tabela abaixo.

Endpoint	Função e dados enviados
`POST /auth/login`	Autentica o usuário. Envia `username` e `password`.
`POST /users/add`	Cria uma conta demonstrativa. Envia `username`, `firstName`, `email` e `password`; o nome de usuário também vai como `firstName` para aparecer na saudação.

- **Fluxo do login:** a tela impede o envio padrão do formulário, lê os campos com `FormData`, remove espaços do usuário e chama `/auth/login`. Com resposta de sucesso, mostra a

saudação usando `firstName` ou `username`; com HTTP 400 ou 401, informa credenciais inválidas.

- **Fluxo do cadastro:** o link “Cadastre-se” abre o modo de cadastro pelo fragmento `#cadastro`. O envio chama `/users/add` e, com sucesso, a resposta da API alimenta o modal de boas-vindas; em caso de falha, o modal não abre e o erro aparece no formulário.
- **Tratamento das respostas:** a função compartilhada `enviarParaApi` centraliza as chamadas: converte os dados em JSON, trata falha de conexão e resposta que não é JSON, e verifica `response.ok`, repassando o status HTTP e a mensagem de erro da API.

Exemplo de requisição de login:

```
{ "username": "emilys", "password": "emilyspass" }
```

Exemplo de requisição de cadastro:

```
{ "username": "jogador-demo", "firstName": "jogador-demo", "email": "jogador@example.com", "password": "senha-de-demonstracao" }
```

[Inserir print – Mensagem “Usuário ou senha inválidos.” após senha incorreta]

3. Organização e tecnologias (principais)

Arquivo ou pasta	Responsabilidade
src/pages/login/Login.jsx	Formulários de login e cadastro, estados da interface, chamadas HTTP à API, mensagens e modal de boas-vindas.
src/pages/login/Login.css	Estilos das mensagens, do estado de carregamento e do modal de boas-vindas.
src/routes/Rotas.jsx	Inclui a tela de login na rota `/login.html` e trata o modo de cadastro por `#cadastro`.
package.json e package-lock.json	Registram as dependências do projeto e os comandos npm.
vite.config.js	Configura a compilação e a geração da versão de produção.

Tecnologias Utilizadas: React, React DOM, Vite, JavaScript (ES6+), Fetch API, API REST DummyJSON, HTML5 semântico, CSS3 e Vercel.

4. Hospedagem e como testar

A plataforma está publicada na **Vercel**, conectada ao repositório público no GitHub. O Vite gera a versão de produção na pasta `dist/`, que é a saída publicada.

Área desenvolvida (Login e cadastro): https://iaragames-store.vercel.app/login.html

Configuração na Vercel	Valor
Framework Preset	Vite
Root Directory	./
Install Command	npm install
Build Command	npm run build
Output Directory	dist

A cada novo commit na branch principal, a Vercel gera uma nova implantação automaticamente.

Como testar online

1. Abra o endereço https://iaragames-store.vercel.app/login.html.
2. **Login válido:** use o usuário `emilys` e a senha `emilyspass` (credencial de teste da DummyJSON); a tela deve mostrar "Olá, Emily!".
3. **Login inválido:** informe uma senha incorreta; a tela deve indicar "Usuário ou senha inválidos.".
4. **Cadastro:** clique em "Cadastre-se", preencha usuário, e-mail e senha e envie; o modal deve mostrar o nome retornado pela API e o aviso de conta demonstrativa.

Como executar localmente

É necessário ter Node.js e npm instalados. No PowerShell do Windows, se o comando npm for bloqueado pela política de execução, use npm.cmd.

1. Clone o repositório e abra o terminal na pasta do projeto.
2. Instale as dependências: `npm install`
3. Inicie o servidor de desenvolvimento: `npm run dev`
4. Abra o endereço informado pelo Vite com a rota da tela, por exemplo: `http://localhost:5173/login.html`

Para gerar e conferir a versão de produção, rode `npm run build` e depois `npm run preview`.

5. Usabilidade, acessibilidade e validação

Usabilidade e experiência do usuário

- Mantém a identidade visual e o cabeçalho do restante da plataforma.
- Reúne login e cadastro na mesma tela, alternando o modo sem trocar de página.

- Dá retorno imediato: carregamento no botão, saudação no sucesso e mensagem clara no erro.
- Avisa de forma transparente que o cadastro é demonstrativo.

Acessibilidade

- Mensagens de erro usam `role="alert"` e a saudação de sucesso usa `role="status"`, para leitores de tela anunciarem o resultado.
- O modal usa `role="dialog"`, `aria-modal="true"` e título associado por `aria-labelledby`, e fecha pela tecla Escape.
- O campo de usuário usa `autocomplete="username"`, facilitando o preenchimento automático.

Validação realizada (09/11/2026)

Verificação	Resultado
`npm run build`	Concluído com sucesso pelo Vite.
Login com emilys / emilyspass na API	HTTP de sucesso; a resposta trouxe `firstName` e `username`.
Login com senha inválida na API	Rejeitado com HTTP 400, compatível com a mensagem de credenciais inválidas.
Cadastro demonstrativo na API	HTTP de sucesso; a resposta trouxe identificador e dados do usuário simulado.
Fluxo de login na tela React	A interface exibiu “Olá, Emily!”.
Fluxo de cadastro na tela React	O modal apareceu com o nome retornado e o aviso de não persistência.

Limitações conhecidas

- **Cadastro não persistente:** o endpoint `/users/add` simula a criação; a conta não pode ser usada depois para entrar.
- **Sessão não persistente:** os tokens retornados no login não são armazenados e as outras páginas não são protegidas.
- **Recuperar senha e “Lembrar de mim”:** ainda sem funcionamento; a recuperação avisa o usuário disso.
- **Uso em produção:** exigiria backend próprio e armazenamento seguro de credenciais; a DummyJSON serve apenas para demonstração.

6. Referências

DummyJSON - Documentação oficial: https://dummyjson.com/docs

DummyJSON - Autenticação: https://dummyjson.com/docs/auth

DummyJSON - Usuários: https://dummyjson.com/docs/users

MDN Web Docs - Fetch API: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API

MDN Web Docs - ARIA: https://developer.mozilla.org/pt-BR/docs/Web/Accessibility/ARIA

React - Documentação oficial: https://react.dev/

Vite: https://vite.dev/

Vercel - Documentação: https://vercel.com/docs